

The Benefits of Using Static Code Analysis for Python



Dicas e Macetes para Análise de Código Python

Como resolver questões de simulado rapidamente



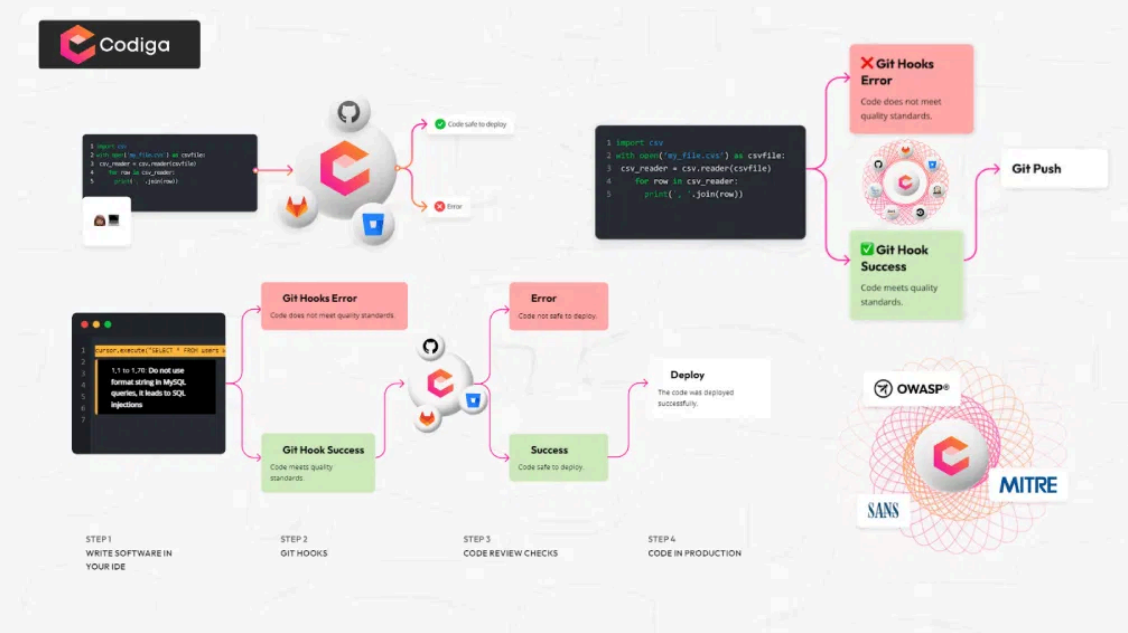
1. Entenda o Objetivo do Código

Antes de analisar linha por linha, identifique o propósito geral do programa:

- Está **ordenando** números?
- Está **calculando** algum valor?
- Está **filtrando** dados?
- Está **comparando** valores?

💡 Dica rápida

Observe os nomes das variáveis e as operações principais. Variáveis como "max", "min", "soma", "contador" já dão pistas sobre o objetivo do código.



2. Rastreamento de Variáveis (Dry Run)

Técnica mais poderosa: **trace manualmente** o código com valores de teste.

```
n1 = int(input()) n2 = int(input()) if n1 > n2: m = n1
else: m = n2 print(m)
```

Teste com diferentes cenários:

- Casos típicos:** Valores normais (n1=5, n2=10)
- Casos de borda:** Valores iguais, zeros, negativos
- Casos específicos:** Baseados nas alternativas

```
Dim y as integer = 3
For x = 1 to 4
    y = y + x
Loop
Console.WriteLine(y)
```

To do this we create a trace table:

x	y	output
1	3	
2	4	
3	6	
4	9	
4	13	13

Linha	n1	n2	m	Condição
1	5	-	-	-
2	5	10	-	-
3-4	5	10	-	5 > 10? Falso
6	5	10	10	-
7	5	10	10	Saída: 10

3. Analise as Condições Lógicas

Os operadores lógicos e de comparação determinam o **fluxo de execução** do programa:

Operadores Lógicos

Combinam condições:

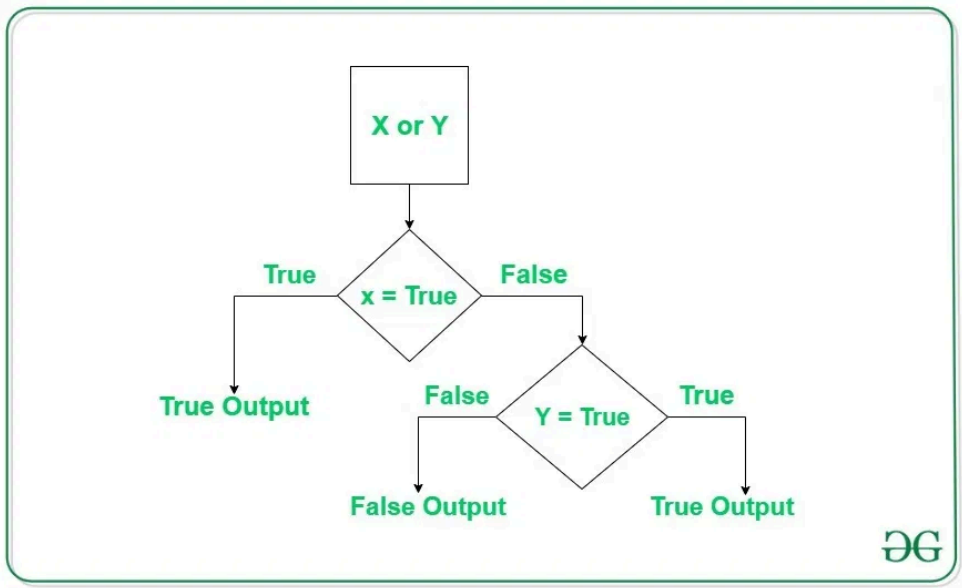
- and** : Ambas as condições devem ser verdadeiras
- or** : Pelo menos uma condição deve ser verdadeira
- not** : Inverte o resultado da condição

Operadores de Comparação

Comparam valores:

- | | |
|-----------------------------|-----------------------------|
| > maior que | < menor que |
| >= maior ou igual | <= menor ou igual |
| == igual a | != diferente de |

Dica: **Teste cada condição separadamente** antes de combiná-las.



Expressão	Resultado	Explicação
5 > 3 and 2 < 4	True	Ambas são verdadeiras
5 > 7 or 2 < 4	True	Segunda é verdadeira
not (5 > 3)	False	Inverte True para False
5 == 5	True	Valores são iguais
5 != 3	True	Valores são diferentes

4. Identifique Padrões e Funções Implícitas

Reconheça padrões comuns de código que implementam funções específicas:

↑ Encontrar o Maior Valor

```
if a > b: max_val = a else: max_val = b # Ou usando a função
built-in max_val = max(a, b)
```

↕ Ordenação Simples

```
if a > b: a, b = b, a # Troca os valores # Agora a ≤ b
```

📊 Acumulador/Contador

```
soma = 0 for i in range(n): soma += valores[i]
```

```
# Exemplo de código com múltiplos padrões def
analise_numeros(lista): # Padrão de inicialização maior
= menor = lista[0] soma = 0 pares = 0 # Padrão de
iteração com acumulador for num in lista: # Padrão de
encontrar máximo if num > maior: maior = num # Padrão
de encontrar mínimo if num < menor: menor = num #
Padrão de soma soma += num # Padrão de contagem
condicional if num % 2 == 0: pares += 1 # Padrão de
cálculo de média media = soma / len(lista) return {
"maior": maior, "menor": menor, "soma": soma, "media":
media, "pares": pares } # Exemplo de uso resultado =
analise_numeros([5, 2, 8, 1, 9, 4]) print(resultado)
```

Dica: Identificar estes padrões acelera a análise de código em questões de simulado.

5. Eliminação de Alternativas

Quando não tiver certeza da resposta, use o método de eliminação:

- 1

Teste com valores simples
Escolha valores fáceis de calcular mentalmente (ex: 5, 10, 0, 1).
- 2

Elimine alternativas obviamente erradas
Se um exemplo simples refuta uma alternativa, ela está errada.
- 3

Teste casos de borda
Valores iguais, zeros, negativos ou extremos podem revelar comportamentos específicos.

 Dica prática

Comece eliminando as alternativas mais improváveis. Às vezes, é mais fácil provar que uma alternativa está errada do que provar que está correta.

Exemplo de Eliminação

Código: `print(max(n1, n2, n3, n4))`

Alternativas:

- a. Exibe o maior dos 4 números.
- ~~b. Exibe o menor dos 4 números.~~

~~✗ Refutado com n1=5, n2=10, n3=15, n4=20 → Saída: 20~~
- ~~c. Exibe a média dos 4 números.~~

~~✗ Refutado com n1=1, n2=2, n3=3, n4=4 → Saída: 4 (média seria 2.5)~~
- ~~d. Exibe a soma dos 4 números.~~

~~✗ Refutado com n1=1, n2=1, n3=1, n4=1 → Saída: 1 (soma seria 4)~~

<pre>01 num = float(input("Enter a number: ")) 02 total = 0 03 04 do 05 total = total + num 06 num = num - 1 07 until num <= 0 08 09 print(total)</pre>	Line Number	num	total	num <= 0
	1	3.5		
	2		0	
	5		3.5	
	6	2.5		
	7			False
	5		6	
	6	1.5		
	7			False
	5		7.5	
	6	0.5		
	7			False
	5		8	
	6	-		

6. Atenção aos Detalhes

Pequenos detalhes podem mudar completamente o comportamento do código:

</> Tipos de Dados

Conversões como `int()`, `float()`, `str()` alteram o comportamento.

Compare:

`5 / 2 = 2.5` vs `5 // 2 = 2`

🔧 Escopo de Variáveis

Variáveis definidas dentro de blocos condicionais podem não estar disponíveis fora deles.

🖨 Saída (`print()`)

Verifique **exatamente** o que está sendo impresso: um valor único, múltiplos valores, ou uma string formatada?

Exemplo: Diferença entre == e is

```
a = [1, 2, 3]
```

```
b = [1, 2, 3]
```

```
c = a
```

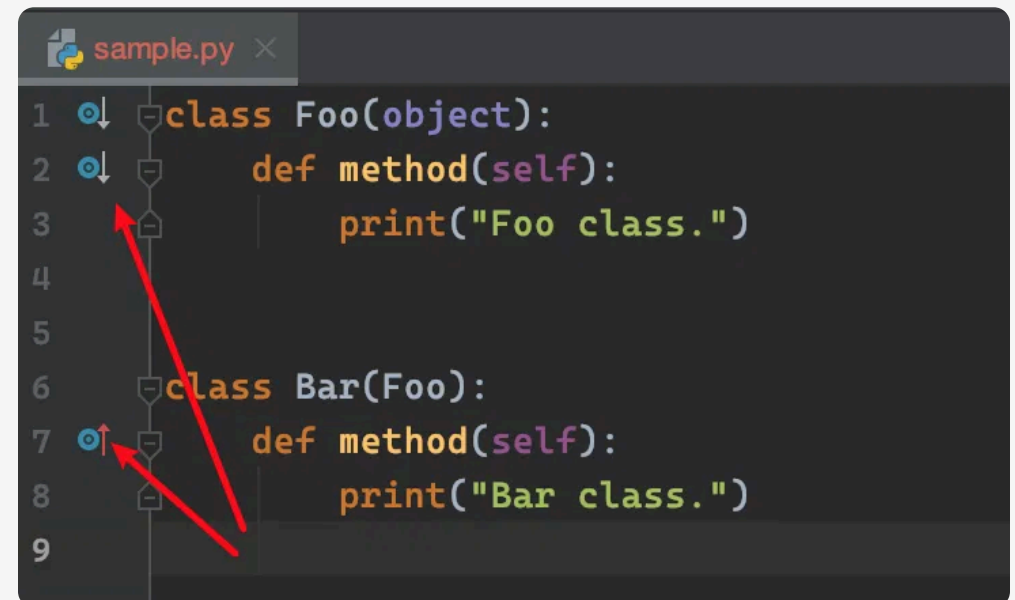
Comparação de valores

```
print(a == b) # True
```

Comparação de identidade

```
print(a is b) # False
```

```
print(a is c) # True
```



```
sample.py x
1 class Foo(object):
2     def method(self):
3         print("Foo class.")
4
5
6 class Bar(Foo):
7     def method(self):
8         print("Bar class.")
9
```

7. Exemplo Prático

Vamos analisar um exemplo real de questão de simulado:

+ Análise passo a passo

- 1 **Entenda o objetivo:** O código compara valores e exibe resultados baseados em condições.
- 2 **Rastreie as variáveis:** Teste com valores como $n1=10$, $n2=5$, $n3=20$ e $n4=15$.
- 3 **Analise as condições:** Observe que $m12$ armazena o maior entre $n1$ e $n2$, e $m34$ o maior entre $n3$ e $n4$.
- 4 **Identifique o padrão:** O código está encontrando o maior número entre os quatro valores de entrada.

Conclusão: O programa exibe o maior número entre os quatro valores de entrada.

```
if m1 > m2:
    print("m1 é maior")
elif m1 < m2:
    print("m1 é menor")
else:
    print("m1 e m2 são iguais")
```

Diagrama de anotações:

- variáveis $m1$ e $m2$ (ponto de partida para a primeira condição)
- $m1$ é maior que $m2$? (condição verdadeira)
- caso contrário,... (condição falsa)
- caso contrário, ... (condição verdadeira)

💡 Dica para questões similares

Quando encontrar códigos com múltiplas comparações, procure identificar se está buscando valores máximos/mínimos ou realizando algum tipo de ordenação.

Dicas Finais

Estratégias para resolver questões de análise de código rapidamente :

🕒 Gerencie seu tempo

Não gaste mais de 2-3 minutos em uma questão difícil. Marque-a e volte depois.

🗑️ Eliminação por contradição

Encontre um único caso que contradiga uma alternativa para eliminá-la.

✍️ Use papel e caneta

Faça o rastreamento manual das variáveis para visualizar o fluxo do programa.

🔍 Procure pistas no enunciado

O enunciado frequentemente contém dicas sobre o que o código deve fazer.

Lembre-se:

A prática é a chave para dominar a análise de código. Quanto mais questões você resolver, mais rápido identificará padrões comuns.



★ Dica de ouro

Quando estiver em dúvida entre duas alternativas, teste com valores extremos ou casos de borda (zeros, negativos, valores iguais) para identificar diferenças sutis no comportamento do código.

✓ Checklist rápido

- Entendi o objetivo do código?
- Testei com pelo menos um exemplo?
- Verifiquei as condições lógicas?
- Identifiquei algum padrão conhecido?
- Eliminei alternativas impossíveis?